02 — Building a Transformer From Scratch

How a transformer block is put together, how to write every gradient by hand and check it, and the non-transformer classics that interviewers ask for.

Reading time: about 45 minutes. You need: modules 00 and 01, or equivalent comfort with tensors and the chain rule. You do not need: any knowledge of deep-learning framework internals.

Forty minutes and a blank file

Picture the interview. You are asked to implement multi-head attention from memory, in forty minutes. You know the formula, so you start typing. At minute thirty the output has the right shape, but the numbers look strange, and you cannot tell why.

This happens to strong candidates all the time. The problem is rarely the maths. It is losing track of which axis is which, and having no way to check the result.

This article gives you a method that avoids both problems. We will build a transformer block one part at a time. For each part, we will write the backward pass by hand and check it numerically. The same method works for the other classic from-scratch questions, which close the article.

Why write gradients by hand when PyTorch computes them for you? Because it forces you to know what every tensor is for. Once you have derived LayerNorm's backward pass yourself, you can explain why LayerNorm helps training.

What you will learn

By the end of this article, you will be able to:

- check any gradient you write, using finite differences;
- describe a pre-norm transformer block, and explain why it replaced post-norm;
- derive the backward pass of LayerNorm, and say what it does geometrically;
- explain how a transformer learns the order of tokens;
- implement logistic regression, k-means++ and AUC without falling into their standard traps.

Section 1 is the working method. Sections 2 to 5 build the transformer block. Section 6 covers the classics. A recap and the exercise briefing follow at the end.

---

1. Write the shapes first, then check every gradient

Two habits make from-scratch work reliable. Write down every shape before writing any code. And have a test that tells you whether a gradient is right.

The shape table

Before coding, list every tensor with named dimensions. Here is the table for multi-head attention. It uses batch size B, sequence length T, model width C, and H heads of size D = C / H.

Tensor	Shape	How it is made
x	B × T × C	the input
qkv	B × T × 3C	one linear layer applied to x
q, k, v	B × H × T × D each	split into three, then into heads
scores	B × H × T × T	q times k transposed, divided by √D, plus mask
weights	B × H × T × T	softmax over the last axis
output per head	B × H × T × D	weights times v
merged output	B × T × C	move the head axis back, then join the heads

The table takes two minutes to write. It turns forty minutes of guessing into a series of small steps, each with a known input shape and output shape.

The riskiest row is the head split. You must reshape `(B, T, 3C)` into `(B, T, 3, H, D)` and then reorder the axes. Reshaping straight to `(B, H, T, D)` also runs, but it mixes positions and heads together. Module 03 plants exactly this bug.

Checking gradients with finite differences

A derivative is the limit of a small difference. So you can estimate any partial derivative by nudging one input a little in each direction:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}$$

Here e_i is a vector with a 1 in position i and zeros elsewhere. This estimate is called a **central difference**, and its error shrinks in proportion to ϵ^2. Compare it with your hand-written gradient, entry by entry. If the two agree, your gradient is almost certainly right.

The choice of ϵ matters more than you might expect. This table shows the largest error when checking the gradient of $\sum_i \sin(x_i)$ in float64:

ε	Largest error
10^{-1}	1.7×10^{-3}
10^{-3}	1.7×10^{-7}
10^{-5}	2.0×10^{-11}
10^{-7}	6.7×10^{-10}
10^{-9}	4.6×10^{-8}

Two effects pull in opposite directions. A large ϵ makes the approximation itself poor. A tiny ϵ makes the two function values almost identical, so subtracting them loses precision to rounding. In float64 the best balance is near $\epsilon = 10^{-5}$.

In float32 the best achievable error in the same test was only about 10^{-5}. That is why gradient checks run in float64, and why a float32 "failure" is often a false alarm. PyTorch's `torch.autograd.gradcheck` performs exactly this test.

Overfit a handful of examples

One more check works on the whole model. Train it on ten examples. A correct model with a correct training loop drives the loss close to zero, because memorising ten examples is easy. If it cannot, something is broken. Module 03 builds a whole debugging method on this test.

> **In the interview:** say the shape table out loud before you type. Finish with one test, such as "changing the last token must not change earlier outputs". A short test is worth more than an extra feature.

---

2. A transformer block: two sublayers, two shortcuts

A transformer is a stack of identical blocks. Each block has two sublayers. Multi-head attention moves information between tokens. An MLP then processes each token on its own. Each sublayer comes with a LayerNorm and a shortcut.

The modern arrangement is called **pre-norm**:

$$h = x + \mathrm{MHA}(\mathrm{LN}_1(x))$$

$$y = h + \mathrm{MLP}(\mathrm{LN}_2(h))$$

Read the first line as: normalise the input, run attention on it, and add the result back to the original input. That addition is the shortcut, called a **residual connection**. The second line does the same with the MLP.

A whole model wraps a stack of these blocks:

```
tokens -> embedding -> [block] x L -> LayerNorm -> linear -> scores
```

The final linear layer produces one score for every word in the vocabulary.

The residual stream

A useful way to picture a transformer is as a stream of vectors flowing from input to output. Each sublayer reads from the stream, computes something, and adds its result back. Nothing is overwritten; contributions pile up. This picture is called the **residual stream**, and most research on interpreting transformers is built on it.

Why pre-norm replaced post-norm

The original 2017 transformer used **post-norm**, which normalises after the addition: $\mathrm{LN}(x + \mathrm{Sublayer}(x))$. Almost every model since GPT-2 uses pre-norm instead.

The reason is how gradients travel. In pre-norm, the path from output to input is an unbroken chain of additions, so a gradient can flow straight down the shortcuts. In post-norm, every addition is followed by a LayerNorm, so the gradient must pass through all of them. Post-norm models are harder to train as a result, and need a careful learning-rate warm-up to avoid diverging early.

A small test shows the imbalance. In a 24-layer stack at initialisation, the last layer of the post-norm version received 3.5 times the gradient of its first layer. In the pre-norm version the ratio was 2.2. The gap is

moderate at this depth. Xiong et al. (2020) show analytically why large gradients near the output are what force post-norm models to warm up.

Pre-norm has a cost as well. Nothing normalises the stream itself, so its size grows with depth. That is why pre-norm models add a final LayerNorm before the output layer. It is also why the projections that write into the stream are often initialised smaller. GPT-2 scales them by $1/\sqrt{2L}$, so that the stream grows no faster than depth allows.

---

3. LayerNorm, and what its gradient does

LayerNorm rescales each token's vector to have mean 0 and variance 1, then applies a learned scale and shift:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(x) = \gamma \odot \hat{x} + \beta$$

Here x is one token's vector of length d. μ and σ^2 are the mean and variance of its d entries. ϵ is a tiny constant that prevents division by zero, and has nothing to do with the finite-difference step of section 1. γ and β are learned vectors of length d.

Why the definition has two steps

The first step is a constraint, and the second one relaxes it. $\hat{x}$ is the normalisation: it is the whole of what "normalise" means here, and with $\gamma = 1$ and $\beta = 0$, which are PyTorch's defaults, $\mathrm{LN}(x)$ is exactly $\hat{x}$.

The constraint on its own is too strict to end with. It forces every token's vector to have mean 0 and variance 1, so no feature could ever be consistently larger than the others, whatever the data says. The second step gives each of the d features back its own scale and offset, learned from the data.

The asymmetry between the two steps is the point. μ and σ^2 are computed per token, while γ and β are shared by every token. So the affine step cannot undo the normalisation: it can say which features should be large in general, but it cannot restore one token's original scale. BatchNorm differs here, because its statistics are per feature, so there γ and β can undo the normalisation exactly.

One warning about the hat. It marks a standardised vector, not one scaled to length 1. Each entry of $\hat{x}$ has variance 1, so the vector itself has length $\sqrt{d}$.

Notice what is being averaged: the features of a single token. Other examples in the batch play no part, and neither do other tokens. That is why LayerNorm works with a batch of one, needs no running statistics, and ignores padding. These properties are why it replaced BatchNorm in sequence models.

A quick sanity check

LayerNorm removes the mean and the scale of its input, so shifting or scaling the input should make no difference. On a random input, `LN(3x + 7)` differed from `LN(x)` by at most 2.1×10^{-5}. The difference is not exactly zero only because ϵ slightly breaks the symmetry.

The backward pass

This derivation is worth doing once, carefully. The difficulty is that μ and σ^2 depend on every entry of x, so every output depends on every input.

Two marks appear in what follows. The hat on $\hat{x}$ marks the normalised vector defined above. A bar means the gradient of the loss with respect to that quantity, as in module 01, and always has the same shape as it. So write $\bar{\hat{x}} = \bar{y} \odot \gamma$ for the gradient arriving at $\hat{x}$. The result is:

$$\bar{x} = \frac{1}{\sqrt{\sigma^2 + \epsilon}}\Big(\bar{\hat{x}} - \text{mean}(\bar{\hat{x}}) - \hat{x} \cdot \text{mean}(\bar{\hat{x}} \odot \hat{x})\Big)$$

Each mean is taken over the d features. Read it one term at a time:

- $\bar{\hat{x}}$ is the incoming gradient.
- Subtracting its mean removes the part that points along the all-ones direction. That direction would shift every feature equally.
- Subtracting $\hat{x}$ times a mean removes the part that points along $\hat{x}$ itself. That direction would stretch the vector.

So LayerNorm projects the gradient away from exactly the two directions it ignores: shifting and scaling. A change in those directions cannot affect the output, so it receives no gradient. That one sentence captures the whole formula.

Why this helps training

There is a practical consequence. Place a linear layer with weights W in front of a LayerNorm. Scaling W up or down does not change the output, so the gradient with respect to W must be orthogonal to W. In one test, the cosine between the two was 4.9×10^{-7}: zero, apart from the effect of ϵ. So the size of W stops affecting the loss, which keeps optimisation better conditioned (module 01, section 3).

A cheaper variant called **RMSNorm** skips the mean subtraction and drops β. It divides x by its root mean square and multiplies by γ. It works about as well, and the Llama family of models uses it.

---

4. The MLP: where each token is processed on its own

The second sublayer is a small two-layer network, applied to each token separately:

$$\text{MLP}(x) = W_2\, \phi(W_1 x)$$

By convention, the hidden layer is four times wider than the model. Every weight matrix holds the product of its two dimensions, so counting a block's parameters is a matter of listing its matrices:

Matrix	Shape	Entries
query, key and value projections	three of $C \times C$	$3C^2$
output projection	$C \times C$	C^2
MLP first layer, W_1	$C \times 4C$	$4C^2$
MLP second layer, W_2	$4C \times C$	$4C^2$
attention subtotal		$4C^2$
MLP subtotal		$8C^2$
block total		$12C^2$

So about two-thirds of a block's parameters sit in the MLP. In GPT-2 small, where $C = 768$, that is 2.36 million numbers in attention and 4.72 million in the MLP. The two LayerNorms add $4C$ more, which is 3,072 numbers, small enough to leave out of the estimate.

Splitting attention into H heads changes none of these counts. The split is a reshape of the same matrices, so heads decide how the width is divided, not how many parameters there are.

The activation function ϕ is usually **GELU**. It behaves like ReLU for large inputs, but it is smooth near zero and passes a small negative response, so units do not switch off for good.

Many recent models use **SwiGLU** instead, a gated variant with a third matrix W_3. It computes $\text{Swish}(W_1 x) \odot (W_3 x)$ and then applies W_2. Three matrices of size $C \times h$ hold $3Ch$ entries, which equals the $8C^2$ above when $h = 8C/3$. So the hidden width shrinks from the usual $4C$ to about $2.67C$, which is the same $4C$ multiplied by $2/3$. The block then keeps the parameter count it had with two matrices.

Llama-2-7B shows the rule in practice. Its model width is 4,096, so $8C/3$ is 10,923. Its configured hidden width is 11,008, which is that figure rounded up to a multiple of 256. Such shapes suit the hardware better.

Two ways of thinking about the MLP are worth knowing. First, it is the only place where each token receives nonlinear processing on its own. Second, it can be read as a **key–value memory**. Each row of W_1 detects a pattern, and the matching column of W_2 is what gets written into the stream when that pattern appears.

---

5. How a transformer knows the order of tokens

Attention on its own has no sense of order. Shuffle the input tokens, and the output comes out shuffled in exactly the same way, with nothing else changed. This property is called **permutation equivariance**. A quick test confirms it: attention applied to a shuffled input matched the shuffled original output to within 8.9×10^{-16}.

So without extra information, "dog bites man" and "man bites dog" look identical to attention. Position has to be added explicitly. Three approaches are common:

- **Learned position embeddings**, as in GPT-2, add a trained vector for each position. They are simple, but they cannot handle positions beyond the training length.
- **Sinusoidal embeddings**, from the original transformer, add fixed sine and cosine patterns at many frequencies.
- **RoPE**, short for rotary position embedding, rotates each query and key by an angle that grows with its position.

RoPE is now the most common choice, and it has an elegant property. Because both vectors are rotated, their dot product depends only on the *distance* between the two positions. A test with two random vectors shows this:

Query position m	Key position n	m − n	Attention score
5	3	2	−1.012220
12	10	2	−1.012220
1002	1000	2	−1.012220
3	5	−2	0.259582

The score is identical whenever $m - n$ is the same. So the model sees relative position for free, with no extra parameters. When people extend a model's context window, they usually adjust RoPE's frequencies, with methods such as NTK-aware scaling or YaRN.

> **In the interview:** "attention is permutation-equivariant, so position must be added; RoPE adds it by rotation, which makes scores depend only on relative position" is a complete short answer.

---

6. The classics: logistic regression, k-means and AUC

Interviewers also ask for a small set of non-transformer implementations. Each one has a single well-known trap.

Logistic regression: compute the logarithm carefully

The gradient of logistic regression is $X^\top(\sigma(Xw) - y)/n$. This is prediction minus target again, for the same reason as in module 01. The trap is the loss, which needs $\log \sigma(z)$. Computed directly, it fails for large negative z:

```
z = torch.tensor([-800., -30., 0., 30.])
torch.log(torch.sigmoid(z))
F.logsigmoid(z)
```

```
naive:   [   -inf, -30.0000, -0.6931,  0.0000]
stable:  [-800.0000, -30.0000, -0.6931, -9.4e-14]
```

$\sigma(-800)$ underflows to exactly 0, and $\log 0$ is $-\infty$. The stable version uses the identity $\log \sigma(z) = -\log(1 + e^{-z})$, rearranged differently for positive and negative z so that no exponential overflows.

K-means: initialise with care

K-means repeats two steps: assign each point to its nearest centre, then move each centre to the mean of its points. The total squared distance never increases, so the algorithm always stops. But it stops at a local optimum, which depends on where it started.

It has three traps:

1. **Initialisation.** Random starting centres can land badly. **K-means++** picks each new centre with probability proportional to its squared distance from the nearest existing centre, which spreads the centres out. On the exercise's three-cluster data, averaged over 20 runs, k-means++ reached a total squared distance of 139.5, against 1,012.4 with random starts.
2. **Empty clusters.** A centre can end up with no points. Handle this explicitly, or you will divide by zero.
3. **Distances.** Compute all point-to-centre distances at once, using $\|x - c\|^2 = \|x\|^2 - 2\,x \cdot c + \|c\|^2$, instead of looping in Python.

The ROC curve

A classifier of this kind outputs a score, not a verdict. Turning the score into a verdict needs a threshold: call everything at or above it positive. The threshold is a free choice, and two rates describe what any particular choice costs.

- The **true positive rate**, also called recall, is the share of real positives caught: TPR $= TP/(TP + FN)$.

- The **false positive rate** is the share of real negatives wrongly caught: FPR $= FP/(FP + TN)$.

The two move together. Lower the threshold and more positives are caught, but so are more negatives. The **ROC curve** plots TPR against FPR as the threshold sweeps from high to low. The name is radar jargon from the 1940s: receiver operating characteristic.

Take four examples with scores 0.1, 0.4, 0.35 and 0.8, where the last two are positive. Only the scores themselves can change the verdict, so five thresholds cover every case:

threshold	TP	FP	FN	TN	FPR	TPR
above all	0	0	2	2	0.00	0.00
≥ 0.8	1	0	1	2	0.00	0.50
≥ 0.4	1	1	1	1	0.50	0.50
≥ 0.35	2	1	0	1	0.50	1.00
≥ 0.1	2	2	0	0	1.00	1.00

Read the last two columns as (x, y) points and they are the curve itself, a staircase from $(0, 0)$ to $(1, 1)$. With finite data it is always a staircase.

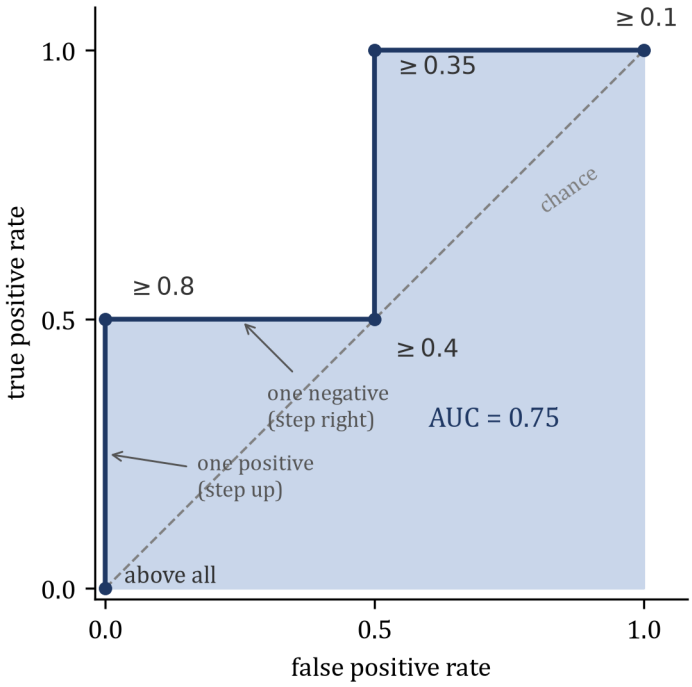


Figure 1: The ROC curve of the four examples. Lowering the threshold walks the curve from $(0, 0)$ to $(1, 1)$: every positive found steps up, every false alarm steps right. The shaded region is the area under the curve.

A curve that hugs the top left corner is a good ranker, because it collects positives before it collects negatives. The diagonal is chance. A curve below the diagonal means the scores are backwards, and flipping their sign would give a good ranker.

AUC: use ranks, not pairs

AUC is the area under that curve. The staircase above encloses two rectangles, 0.5×0.5 and 0.5×1, so the area is 0.75. The number has a second reading, which is the one worth remembering: it is the probability that a randomly chosen positive example scores higher than a randomly chosen negative one. A perfect ranker scores 1, chance scores 0.5, and a ranking that is exactly backwards scores 0.

That second reading suggests comparing every pair, which costs $O(n^2)$. The standard answer uses ranks instead, which costs $O(n \log n)$ for the sort.

Check the equivalence by hand on the same four examples.

1. Sort the scores: 0.1, 0.35, 0.4, 0.8. Their ranks are 1, 2, 3 and 4.
2. The positive examples, 0.35 and 0.8, have ranks 2 and 4. Their ranks add up to 6.
3. Subtract the smallest rank sum two positives could have, which is $1 + 2 = 3$. That leaves 3.
4. Divide by the number of positive–negative pairs, which is $2 \times 2 = 4$. The AUC is 0.75.

Comparing all four pairs directly agrees: three of the four pairs are ordered correctly. The general formula is known as the Mann–Whitney statistic:

$$\mathrm{AUC} = \frac{\sum_{i \in \mathrm{pos}} r_i \; - \; n_p(n_p + 1)/2}{n_p \, n_n}$$

Here r_i are the ranks of the positive examples, and n_p and n_n count the positives and negatives. Tied scores share the average of their ranks.

A good follow-up question is when AUC is the wrong metric. With heavy class imbalance it can look excellent while the model is useless, because the false positive rate divides by the huge number of negatives. Here is the effect, measured on 100 positives and 100,000 negatives drawn two standard deviations apart:

metric	value
AUC	0.934
false positive rate at 90% recall	0.215
false positives at that threshold	21,460
precision at that threshold	0.004
average precision	0.067

The AUC says the model ranks well, and it does. The precision says a reviewer working the flagged queue would find one real case in 240. Average precision, the area under the precision–recall curve, is the metric that reports this, and 0.067 still beats its 0.001 baseline. Report it instead whenever positives are rare and each false positive costs something.

Convolution, briefly

For a convolution with input size n, kernel size k, padding p and stride s, the output size is $\lfloor (n + 2p - k)/s \rfloor + 1$. For example, $n = 32$, $k = 3$, $p = 1$ and $s = 2$ give an output of size 16. The gradient with respect to the input is itself a convolution, using the flipped kernel. Fast implementations unfold the input into one large matrix, a trick called im2col, and then use a single matrix multiplication. That is fast but uses a lot of memory.

Recap

- Write the shape table first. Check every gradient with central differences, in float64.
- A pre-norm block computes $x + \mathrm{MHA}(\mathrm{LN}(x))$, then $h + \mathrm{MLP}(\mathrm{LN}(h))$. The shortcuts carry gradients straight through, which is why pre-norm trains more easily.
- LayerNorm's gradient removes the two directions LayerNorm ignores: shifting and scaling.
- About two-thirds of a block's parameters live in the MLP.
- Attention is permutation-equivariant. RoPE adds position so that scores depend only on distance.
- For the classics: compute log-probabilities stably, seed k-means with k-means++, and compute AUC from ranks.

The exercise: a numpy transformer that learns

What you will build. A complete one-block transformer language model, written in numpy, with every backward pass written by you. You build it one layer at a time: a linear layer, LayerNorm, GELU, softmax, multi-head attention and the full block. Finite differences check each layer as you go. Then you implement k-means++ and AUC.

The model is trained on a **copy task**. Each sequence is 8 random tokens followed by the same 8 tokens again. The second half can only be predicted by looking back exactly eight positions. So if the model learns the task, your attention implementation works.

Time and setup. Allow four to five hours. The setup is the same as for module 00. Your code uses only numpy; the reference solution also uses PyTorch, as an independent check.

How to run it. Open `02-ml-coding-from-scratch/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

How the checks work. They work as in module 00. Before you start, each step reports that it is not implemented yet:

```
[FAIL] step 0  -- NotImplementedError:

0/1 passed.  Still failing: step 0
```

Step 0 builds the finite-difference checker. From step 1 onwards, the checks use it to compare each of your backward passes with a numerical estimate.

Step map. This table links each step to the part of the article it practises.

Step	You write	The idea it practises	Section
0	`numerical_grad`	central differences	1
1	`Linear`, forward and backward	the matrix-multiplication gradient	1
2	`LayerNorm`, then an orthogonality check	the LayerNorm gradient	3
3	`GELU`, `softmax`, `softmax_backward`	activations and a stable softmax	4
4	`MultiHeadAttention`	the shape table	1
5	`TransformerBlock`	pre-norm and residual connections	2
6	`TinyLM` and `Adam`, then training	the whole model	2
7	`kmeans_pp_init`, `kmeans`, `auc`	the classics	6

When you are done. The model predicts the second half of the copy task with 100% accuracy. Its loss settles near 1.12, not near zero, and that is correct.

The reason is the first half. Each sequence gives 15 next-token predictions, and 7 of them fall in the random first half. There, the best any model can do is a loss of $\ln 11$, since each token is one of 11 equally likely values. So the best possible average loss is $\frac{7}{15} \ln 11 = 1.119$. The reference model reaches 1.124.

Step 6b then asks you to print the attention weights. You should see a sharp diagonal band: each token in the second half looking back exactly eight positions.

If you get stuck.

- Compare your shapes with the table in section 1.
- If a gradient check fails, test that layer alone on the smallest input you can, such as a $1 \times 2 \times 4$ tensor.
- In the block's backward pass, remember that a residual connection sends the gradient down both paths. The two contributions must be added together.
- If you are still stuck, `solution.py` has a complete reference, and compares every gradient with PyTorch.

Want more? The notebook ends with four stretch goals: replace LayerNorm with RMSNorm, add RoPE, add gradient checkpointing, and write a 2-D convolution using im2col.

Going deeper

Start here

- Karpathy, *nanoGPT* — https://github.com/karpathy/nanoGPT. About 300 lines of clean, production-quality GPT code. After the exercise, read `model.py` from top to bottom and compare it with your own.

Building along with others

- Karpathy, *Let's build GPT: from scratch, in code, spelled out* — https://www.youtube.com/watch?v=kCc8FmEb1nY. A video that builds a transformer step by step. The best companion to this module.
- Rush (2018), *The Annotated Transformer* — https://nlp.seas.harvard.edu/annotated-transformer/. The original paper, interleaved with working code.
- Bloem (2019), *Transformers from Scratch* — https://peterbloem.nl/blog/transformers. A gentle written explanation of self-attention from first principles.

The architecture

- Vaswani et al. (2017), *Attention Is All You Need*, NeurIPS. arXiv:1706.03762. The original transformer, which used post-norm.
- Radford et al. (2019), *Language Models are Unsupervised Multitask Learners* (GPT-2). The pre-norm, decoder-only design that most models now follow.
- Xiong et al. (2020), *On Layer Normalization in the Transformer Architecture*, ICML. arXiv:2002.04745. Explains analytically why post-norm needs warm-up. Read it if you want to argue for pre-norm rather than assert it.
- Ba, Kiros & Hinton (2016), *Layer Normalization*. arXiv:1607.06450. The original LayerNorm paper.
- Zhang & Sennrich (2019), *Root Mean Square Layer Normalization*, NeurIPS. arXiv:1910.07467. RMSNorm.
- Su et al. (2021), *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864. The paper that introduced RoPE.

- Shazeer (2020), *GLU Variants Improve Transformer*. arXiv:2002.05202. Two pages, and the reason many models have three MLP matrices.
- Geva et al. (2021), *Transformer Feed-Forward Layers Are Key-Value Memories*, EMNLP. arXiv:2012.14913. The key–value memory view of the MLP from section 4.

The classics

- Arthur & Vassilvitskii (2007), *k-means++: The Advantages of Careful Seeding*, SODA. Proves that k-means++ is within a factor of $O(\log k)$ of optimal, in expectation.
- Fawcett (2006), *An Introduction to ROC Analysis*, Pattern Recognition Letters 27(8). Section 7 covers the rank formula and how to handle ties.
- Saito & Rehmsmeier (2015), *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*, PLoS ONE 10(3). When and why precision–recall beats ROC.
- Goodfellow, Bengio & Courville (2016), *Deep Learning*, MIT Press. Free online. Chapter 9 covers convolution arithmetic and its backward pass.

Next: module 03 hands you this same architecture with eight bugs planted in it.